SAMSUNG

Samsung Innovation Campus

Generative AI Module

Together for Tomorrow!
Enabling People
Education for Future Generations

Notas de Clase

Generative AI Module

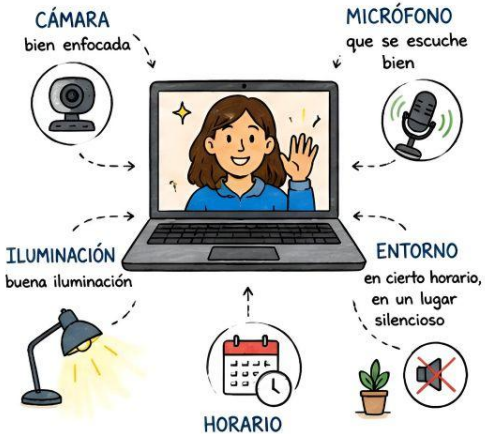
OBJETIVOS DEL CURSO

1 Se lleven como razonar un problema y saber cuando usar AI y cuando no.



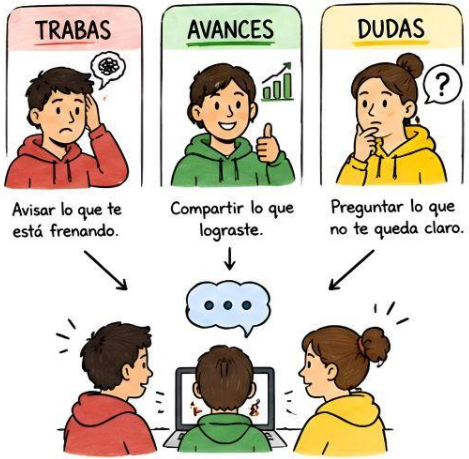
Primero pensar, después decidir.

2 Organizar su vida para tener meets, eso implica tener tu carta de presentacion en buenas condiciones:



Tu meet es tu carta de presentación. ¡Cuidala!

3 Poder comunicar trabas, avances y dudas a tiempo, es preferible equivocarse aca que con una deadline.



Comunicar a tiempo = mejores soluciones, menos estrés.

PENSAR. ORGANIZAR. COMUNICAR. = CRECER JUNTOS


DuckCredit AI — Sistema inteligente de scoring crediticio
Propuesta de desarrollo de un MVP de Machine Learning
BANCO
SOLICITUD DE CRÉDITO
Un futuro más brillante también cuenta contigo
1 ENTRADA
Datos del solicitante
Ingresos mensuales
Ej. $25,000
Deudas existentes
Ej. $8,000
Historial de pagos
Ej. 24/24 pagos
Antigüedad laboral
Ej. 3 años
Monto solicitado
Ej. $50,000
2 PROCESO
Modelo de Machine Learning
DuckCredit AI
Análisis de datos
Modelo predictivo
Evaluación de riesgo
Decisión explicable
MODELOS
EVALUACIÓN
IMPACTO
GOOD DATA BETTER PEOPLE
De datos a oportunidades
3 SALIDA
Score y categoría de riesgo
Riesgo bajo
Score: 0.85
Probabilidad de impago: 15%
Riesgo medio
Score: 0.60
Probabilidad de impago: 40%
Riesgo alto
Score: 0.25
Probabilidad de impago: 75%
Decisiones de crédito más justas, rápidas y basadas en datos.

DuckCredit AI — Arquitectura propuesta, alcance y cronograma

Arquitectura propuesta para el MVP



Cronograma estimado

1	2	3	4	5
Assessment y EDA	Feature Engineering	Model Training & Evaluation	API & Deployment	Testing & Documentation
1 semana	1 semana	1 semana	1 semana	1 semana

Total — 5 semanas

Entregables comprometidos

- Pipeline reproducible de ML
- Modelo entrenado
- API REST desplegada
- Informe de métricas
- Documentación técnica

DuckCredit AI — Technical Implementation Report

Documentación del modelo y evaluación de rendimiento



1 Dataset y preparación

- **10.000 solicitudes** crediticias simuladas
- **5** variables predictoras
- Variable objetivo: incumplimiento (0/1)
- Split: **80%** entrenamiento / **20%** prueba

Pipeline de preparación de datos

Imputación de faltantes → Codificación → Escalado → Pipeline de predicción

2 Modelos evaluados

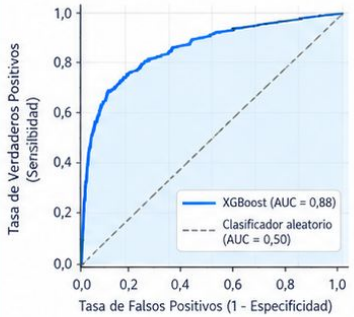
Modelo	ROC-AUC	F1-score
Logistic Regression	0,79	0,68
Random Forest	0,85	0,74
XGBoost	**0,88**	**0,77**

Modelo seleccionado



3 Resultados del MVP

Curva ROC — Modelo seleccionado (XGBoost)



- ROC-AUC: **0,88**
- F1-score: **0,77**
- API preparada para inferencia
- Modelo almacenado en Cloud Storage

Limitaciones

- Datos sintéticos
- Métricas ilustrativas
- Falta evaluación de sesgos y calibración



DuckCredit AI

DEMO — Synthetic Data & Illustrative Metrics

DuckCredit AI — API Reference & Technical Handover

DUMMY API — Educational Example

From model to real impact!

Duck Bank App
User applies for a loan

→

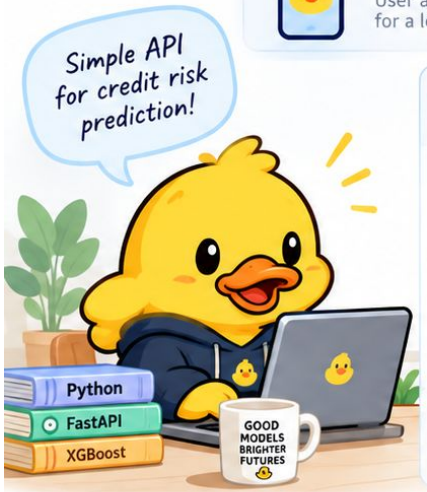
FastAPI
API receives data and calls model

→

XGBoost
ML model predicts default risk

→

JSON
Returns prediction in JSON format



POST `/predict`

Predicts the probability of loan default based on applicant data.

Request Content-Type: application/json

JSON body example:

```
{
    "income": 2500,
    "debt": 500,
    "late_payments": 1,
    "employment_years": 3,
    "loan_amount": 4000
}
```

Response (200 OK)

Returns the model prediction result.

JSON response example:

```
{
    "default_probability": 0.18,
    "risk_level": "low",
    "model_version": "1.0.0"
}
```

GET `/health`

Check if the API and model are running.

Response (200 OK)

```
{
    "status": "ok",
    "model_version": "1.0.0"
}
```

Technical Handover

All materials needed to run, reproduce and maintain the model in production.

Source code
FastAPI app, model inference and utilities.
`/src`

Model artifact
Trained XGBoost model file.
`/models/duckcredit.pkl`

Deployment guide
Steps to deploy locally or on the cloud.
`/docs/deployment.md`

Testing results
Unit tests and sample predictions.
`/tests/`

Documentation
API reference, examples and usage notes.
`/docs/api_reference.md`

SLIDE 1

AUTOMATIZAR, EJECUTAR Y ORGANIZAR

Tres herramientas para proyectos de IA/ML



Objetivo de la clase

Conocer y aplicar tres herramientas clave que nos permiten construir soluciones completas: automatizar procesos, ejecutar código de forma remota y organizar el trabajo en equipo.

DESAFÍO 1: MAKE + CLAUDE

Conectar Claude con Make mediante Skills para crear y ejecutar una automatización.

- Conectar servicios y aplicaciones.
- Diseñar un escenario con un disparador y una acción.
- Usar lenguaje natural con Claude para construir y modificar el escenario.
- Ejecutar una prueba y verificar la entrada y salida.

Resultado: automatización funcional.

DESAFÍO 2: APIFY + API

Crear un Actor en Python y utilizar solicitudes HTTP POST y GET para ejecutarlo y recuperar sus resultados.

- Crear un Actor de Python en Apify.
- Enviar un JSON mediante POST.
- Consultar el estado de la ejecución.
- Recuperar los resultados mediante GET.

Resultado: Actor funcional y respuestas HTTP.

DESAFÍO 3: TRELLO + KANBAN

Organizar un proyecto utilizando un tablero Kanban para visualizar tareas, responsables y avances.

- Crear un tablero con tres listas (Por hacer, En proceso, Finalizado).
- Agregar al menos cinco tarjetas.
- Asignar responsables, fechas de entrega y etiquetas.
- Mover las tarjetas para simular el avance del proyecto.

Resultado: tablero de Trello con tareas y estados definidos.

SLIDE 1

DESAFÍO 1: CONECTAR MAKE CON CLAUDE MEDIANTE SKILLS

Tiempo estimado: **15 minutos**

Objetivo

Conectar Claude con Make utilizando Make Skills para diseñar y ejecutar una automatización mediante instrucciones en lenguaje natural.



Consigna

1. Ingresar a https://skills.make.com/ y revisar cómo funcionan los Skills de Make.
2. Configurar Claude para utilizar las instrucciones del Skill de Make.
3. Crear un escenario sencillo en Make con un disparador y una acción.
4. Solicitarle a Claude que explique, construya o modifique el escenario siguiendo las instrucciones del Skill.
5. Ejecutar una prueba y verificar los datos de entrada y salida.

Ejemplo de automatización: Saludo estructurado

1. Disparador	2. Procesamiento	3. Respuesta
Webhooks Custom webhook	**Tools** Set multiple variables	**Webhooks** Webhook response
Recibe un JSON con nombre y mensaje.	Genera saludo, transforma el mensaje y cuenta caracteres.	Devuelve una respuesta HTTP en formato JSON.

Ejemplo de entrada (INPUT)

```
{
  "nombre": "Ana",
  "mensaje": "Hola Make"
}
```

Resultado esperado (OUTPUT)

```
{
  "saludo": "Hola, Ana",
  "mensaje_procesado": "HOLA MAKE",
  "caracteres": 9,
  "procesado_en": "2024-10-14T12:30:00Z"
}
```

Flujo IPO

INPUT	PROCESS	OUTPUT
JSON recibido por un webhook.	Módulos de Make configurados y asistidos por Claude.	Respuesta HTTP con datos transformados.

Entregable

- Escenario funcional en Make.
- Captura de ejecución.
- Respuesta HTTP obtenida.
- Evidencia de la interacción con Claude.
- Breve explicación del papel de Claude y del Skill.

Pregunta de reflexión

¿Qué diferencia existe entre utilizar Claude para generar instrucciones y utilizar Make para ejecutar una automatización?

SLIDE 2

DESAFÍO 2: API REST CON APIFY — POST Y GET

Crear un Actor en Python, ejecutarlo mediante la API de Apify y recuperar sus resultados.

Tiempo estimado: 15–20 minutos

1 Crear el Actor

1. Ingresar a **console.apify.com**
2. Develop new → Python
3. Host on Apify (editor web)
4. Asignar un nombre (ej.: hola-actor)



2 Configurar input_schema.json

Editar .actor/input_schema.json para recibir un parámetro **name**.

```
{
  "title": "Hola Actor",
  "type": "object",
  "schemaVersion": 1,
  "properties": {
    "name": {
      "title": "Nombre",
      "type": "string",
      "description": "Nombre
        a saludar",
      "editor": "textfield",
      "prefill": "Claude"
    }
  },
  "required": ["name"]
}
```

3 Programar src/main.py

Generar el mensaje y guardarlo en el dataset de Apify.

```
from apify import Actor

async def main() -> None:
  async with Actor:
    data = await Actor.get_input()
      or {}
    name = data.get("name", "Mundo")
    Actor.log.info(f"Procesando
    nombre: {name}")
    await Actor.push_data({
      "message": f"¡Hola, {name}!"
    })
```

4 Compilar y probar

1. Hacer clic en **Save & Build**.
2. Ejecutar con **Start**.
3. Revisar el resultado en la pestaña **Output**.



5 Obtener credenciales

1. Token: Settings > API & Integrations > Personal API **tokens**.
2. ID del Actor: USUARIO~hola-actor (con virgulilla).

Necesitarás:

- TOKEN de API
- ID del Actor (**USUARIO~hola-actor**)

6 Solicitudes HTTP a la API de Apify

1 POST — Iniciar ejecución

https://api.apify.com/v2/acts/USUARIO~hola-actor/runs

```
curl -X POST "https://api.apify.com/v2/acts/USUARIO~hola-actor/runs" \
  -H "Authorization: Bearer TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name": "Claude"}'
```

2 GET — Consultar estado

https://api.apify.com/v2/actor-runs/RUN_ID

```
curl "https://api.apify.com/v2/actor-runs/RUN_ID" \
  -H "Authorization: Bearer TOKEN"
```

Repetir hasta que el campo **status** sea SUCCEEDED.

3 GET — Recuperar resultado

https://api.apify.com/v2/datasets/DATASET_ID/items

```
curl "https://api.apify.com/v2/datasets/DATASET_ID/items" \
  -H "Authorization: Bearer TOKEN"
}
```

Flujo IPO

INPUT	PROCESS	OUTPUT
JSON con el nombre enviado mediante POST.	Actor de Python ejecutado en Apify.	Resultado almacenado y recuperado mediante GET.

Resultado esperado

```
[
  {
    "message": "¡Hola, Claude!"
  }
]
```

Entregable

- Actor funcional en Apify.
- Código Python y input_schema.json.
- Evidencias de POST y GET.
- RUN_ID, DATASET_ID y estado final.

Pregunta de reflexión

¿Por qué POST se utiliza para iniciar una ejecución y GET para consultar su estado y recuperar los resultados?

Documentación: https://docs.apify.com/api/v2

SLIDE 3

DESAFÍO 3: INTRODUCCIÓN A TRELLO
GESTIÓN DE TAREAS CON KANBAN

Trello

Organiza
Visualiza
Colabora
Avanza

Objetivo
Conocer Trello como herramienta de organización y seguimiento de proyectos, utilizando la metodología Kanban para visualizar el flujo de trabajo.

Tiempo estimado
15 minutos

Consigna
1. Ingresar a trello.com y crear una cuenta o iniciar sesión.
2. Crear un tablero para un proyecto de automatización o Machine Learning.
3. Organizar el tablero en tres listas: Por hacer, En proceso y Finalizado.
4. Crear al menos cinco tarjetas que representen tareas concretas del proyecto.
5. Asignar responsables, fechas de entrega, etiquetas de prioridad y checklists.
6. Mover las tarjetas entre listas para simular el avance del proyecto.



Ejemplo de tarjeta

Realizar EDA

Ana
14 oct
Alta

Checklist

- [ ] Cargar dataset e identificar tipos de datos.
- [ ] Calcular valores nulos.
- [ ] Graficar distribuciones.
- [ ] Analizar correlaciones.
- [ ] Registrar tres conclusiones.

Flujo IPO

INPUT	PROCESS	OUTPUT
Tareas, responsables, prioridades y fechas.	Organización, asignación y seguimiento mediante Kanban.	Tablero visual con el estado actualizado del proyecto.

Entregable
Tablero de Trello con al menos cinco tarjetas, responsables, etiquetas de prioridad, checklists y estados definidos.



Pregunta de reflexión
¿Cómo ayuda Kanban a identificar tareas, pendientes, cuellos de botella y responsabilidades dentro de un equipo?



Slide 08. Variables categóricas de alta cardinalidad: target encoding y suavizado bayesiano

① El problema

Variables categóricas con muchas categorías (ID de usuario, producto, ubicación) no se pueden one-hot encodear: generan miles de columnas, modelos inestables y riesgo de sobreajuste.

② La idea

Reemplazar cada categoría por una estimación de la media del target: $\hat{\mu}_c = \frac{1}{n_c} \sum_{i} y_i$

En clasificación es la tasa: $\hat{p}_c = \frac{1}{n_c} \sum_{i} y_i$ (proporción de clase positiva).

③ El problema de la media cruda

Con pocas observaciones, la media es ruidosa. Ejemplo (clasificación):

Categoría	n	# positivos	Tasa observada
A	100	30	0,30
B	10	5	0,50
C	2	2	1,00

La categoría C parece perfecta, pero es solo por tener muy pocos datos.

④ Suavizado bayesiano

Combinamos la media de la categoría con la media global (prior) según cuántos datos tiene:

$$\tilde{\mu}_c = \underbrace{\frac{n_c}{n_c + \alpha}}_{\text{información de la categoría}} \hat{\mu}_c + \underbrace{\frac{\alpha}{n_c + \alpha}}_{\text{información global (prior)}} \mu_0$$

α controla cuánto confiamos en la media global. Con pocos datos (n_c pequeño), $\tilde{\mu}_c$ se acerca a μ_0. Con muchos datos, $\tilde{\mu}_c$ se acerca a $\hat{\mu}_c$.

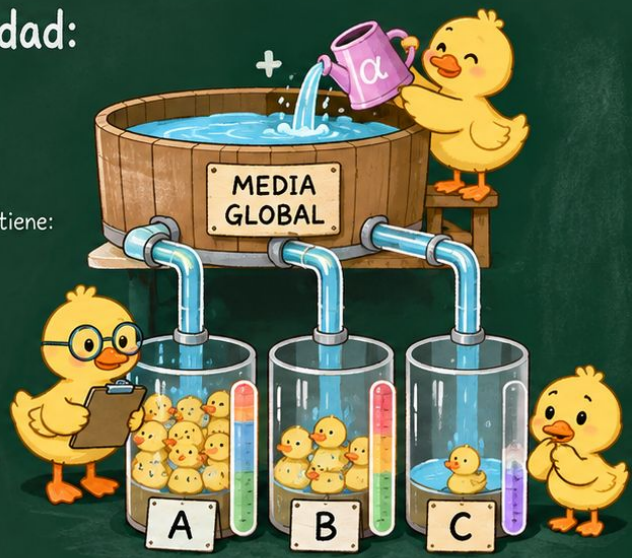
⑤ Ejemplo numérico

Supongamos $\mu_0 = 0{,}20$ y $\alpha = 10$.

Categoría	n	Tasa observada	Tasa suavizada
A	100	0,30	0,29
B	10	0,50	0,35
C	2	1,00	0,33

⑥ Buenas prácticas

- Calcular el encoding solo con el train (evita leakage).
- Usar validación cruzada out-of-fold para obtener valores sin "ver" el target de la misma fila.
- Elegir α por validación (puede variar por dataset).
- Para regresión, el mismo esquema aplica a la media del target.



Puente P&E → ML

- Estimación bayesiana = combinar información muestral y un prior.
- Sesgo-varianza: el suavizado reduce la varianza de estimadores con pocos datos.
- En ML, és esto se usa como una transformación de variables para mejorar la generalización.

09 Feature Engineering: transformaciones que cuentan

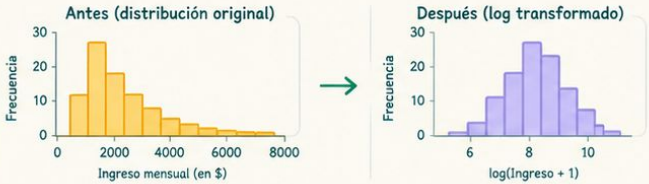
No se trata de aplicar transformaciones por receta, sino de usar lo que vimos en el EDA para mejorar la relación entre las variables y el modelo.

1 Transformaciones comunes y su sentido

Transformación	¿Qué problema resuelve?	Ejemplo
Log / Box-Cox	Reduce asimetría y estabiliza la varianza.	$x' = \log(x + 1)$
Binarización	Convierte a categorías (útil en reglas o modelos simples).	$x' = 1$ si $x > 0$, 0 si no
Escalado	Evita que unas variables dominen a otras.	$z = (x - \mu) / \sigma$
Interacciones	Modela relaciones no lineales o efectos combinados.	$x_1 \cdot x_2$, x_1^2

2 Ejemplo: transformar una variable asimétrica

Supongamos que tenemos el ingreso mensual (en $) de los patitos.



3 ¿Cómo decidir qué transformación usar?

1. **Mirá el EDA:**
   - ¿Hay asimetría? → log o Box-Cox.
   - ¿Hay muchas categorías? → binarización o target encoding.
   - ¿Hay variables en distintas escalas? → escalado.
   - ¿Se observan relaciones no lineales? → interacciones.
2. **Probá y compará:**
   Evaluá el cambio en la distribución y en la performance del modelo (validación).

Idea clave

Las transformaciones no "arreglan" todos los problemas. La elección depende del patrón que vimos en el EDA y del modelo que vamos a usar.

4 Puente P&E → ML

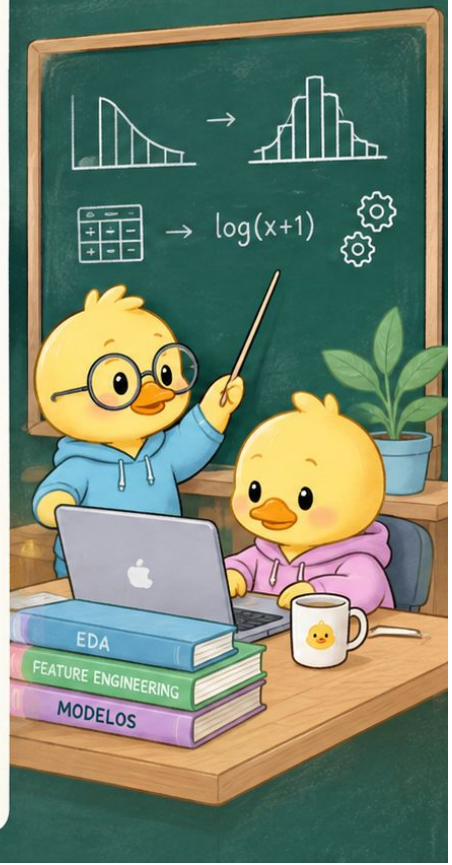
El EDA nos muestra la forma de la distribución y las relaciones entre variables. Esa información guía las transformaciones (feature engineering), que mejoran la capacidad del modelo para aprender.

EDA (patrones) → Feature Engineering (transformaciones) → Mejor modelo (más estable y preciso)



Puente P&E → ML

El EDA nos muestra la forma de la distribución y las relaciones entre variables.
Esa información guía las transformaciones (feature engineering), que mejoran la capacidad del modelo para aprender.



Slide 10

Feature Engineering: unir, crear y seleccionar

De variables crudas a representaciones más informativas

1. Combinar información

- Crear variables que capturen relaciones relevantes entre columnas.
- Ejemplos:
  - ratios: ingreso / gasto
  - diferencias: fecha_(hoy) – fecha_(evento)
  - interacciones: $X_1 \times X_2$

2. Variables de tiempo

- Convertir fechas en información útil.
- Componentes: año, mes, día, día de la semana, estacionalidad.
- Ejemplos:
  - antigüedad en días
  - es fin de semana
  - seno y coseno para estacionalidad

3. Variables categóricas

- Codificar según cardinalidad.
  - Baja: one-hot encoding.
  - Media/alta: target encoding con suavizado o agrupación.
  - Ordinales: respetar el orden.

4. Variables de texto o libre

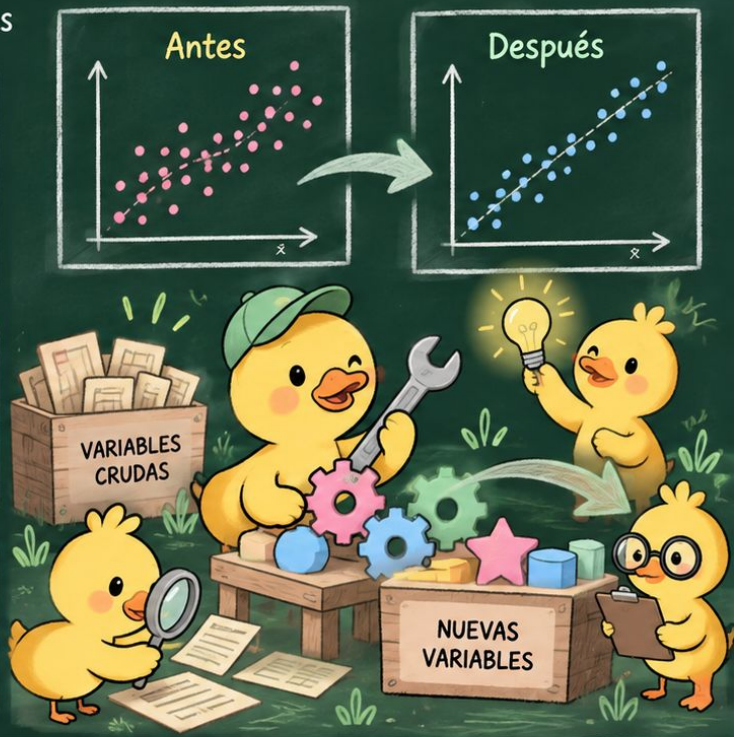
- Convertir texto en representaciones numéricas.
  - Bag of words / TF-IDF.
  - Embeddings.
  - Variables derivadas (longitud, conteos).

5. Seleccionar y reducir

- Eliminar variables irrelevantes o redundantes.
- Métodos: correlación, VIF, selección basada en modelos (Lasso, árboles).
- Objetivo: mejorar generalización y simplicidad del modelo.

Ejemplo integrado

Característica cruda	Nueva variable
fecha_compra	antigüedad_días
ingreso, gasto	ratio_ingreso_gasto
edad, ingreso	interacción_edad_ingreso
descripción	embedding_texto



Puente P&E ⟶ ML

Transformar y seleccionar variables es aplicar funciones a variables aleatorias para construir nuevas variables con mayor poder explicativo de Y.

En P&E estudiamos transformaciones y correlaciones.
En ML usamos estas herramientas para que el modelo aprenda mejor y generalice.

11. El tiempo cambia las reglas

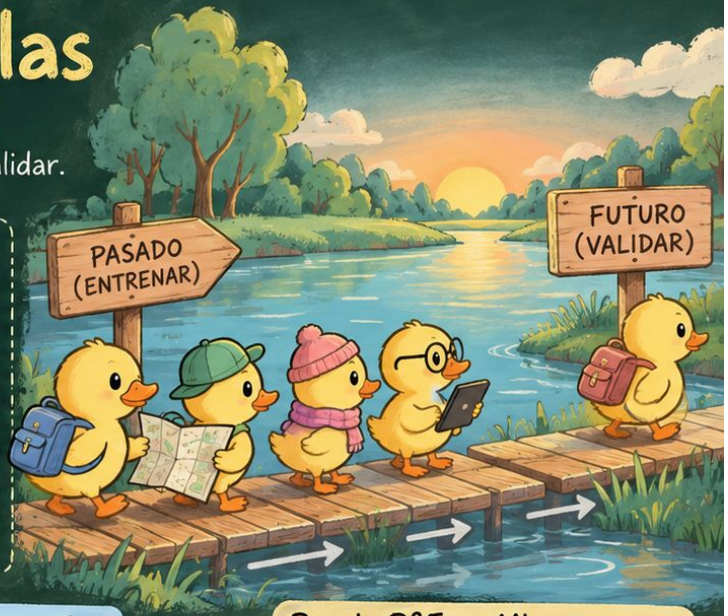
En datos temporales, las observaciones no son i.i.d. y la distribución puede cambiar. Debemos respetar el orden temporal al entrenar y validar.

¿Qué pasa en una serie temporal?

- Las observaciones están **ordenadas** en el tiempo.
- Puede haber **autocorrelación** y **estacionalidad**.
- La distribución puede **cambiar** (tendencia, nivel, varianza).
- No se puede mezclar pasado y futuro al entrenar.

Ejemplo: alimento diario vs. peso



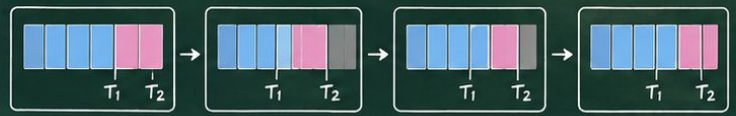


Estrategia de validación

- **Split temporal**: una sola división train (pasado) / test (futuro).
- Validación en ventana deslizante (walk-forward).
- Usar solo información disponible hasta el momento t.
- Evaluar estabilidad del modelo en diferentes períodos.

Validación en ventana deslizante (walk-forward)

1. Entrenar en $1 \ldots T_1$
2. Validar en $T_1{+}1 \ldots T_2$
3. Avanzar la ventana
4. Repetir



Puente P&E → ML

- Las series temporales violan el supuesto i.i.d. de muchas técnicas.
- Conceptos de P&E como autocorrelación, estacionalidad y cambio de distribución explican por qué el tiempo importa.
- La validación temporal es una forma de estimar el error fuera de muestra bajo las condiciones reales de uso.

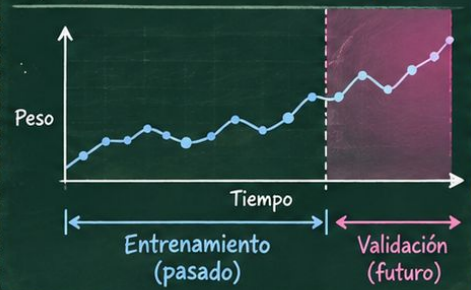
11. El tiempo cambia las reglas

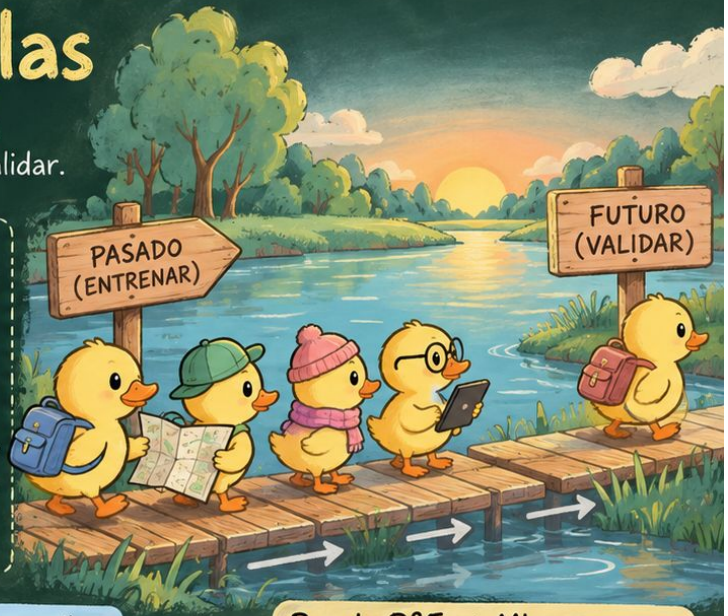
En datos temporales, las observaciones no son i.i.d. y la distribución puede cambiar. Debemos respetar el orden temporal al entrenar y validar.

¿Qué pasa en una serie temporal?

- Las observaciones están **ordenadas** en el tiempo.
- Puede haber **autocorrelación** y **estacionalidad**.
- La distribución puede **cambiar** (tendencia, nivel, varianza).
- No se puede mezclar pasado y futuro al entrenar.

Ejemplo: alimento diario vs. peso



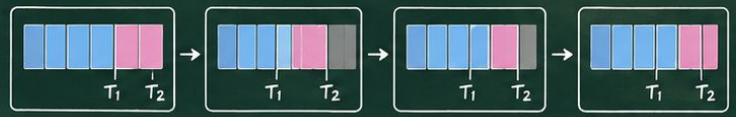


Estrategia de validación

- **Split temporal**: una sola división train (pasado) / test (futuro).
- Validación en ventana deslizante (walk-forward).
- Usar solo información disponible hasta el momento t.
- Evaluar estabilidad del modelo en diferentes períodos.

Validación en ventana deslizante (walk-forward)

1. Entrenar en $1 \ldots T_1$
2. Validar en $T_1{+}1 \ldots T_2$
3. Avanzar la ventana
4. Repetir



Puente P&E → ML

- Las series temporales violan el supuesto i.i.d. de muchas técnicas.
- Conceptos de P&E como autocorrelación, estacionalidad y cambio de distribución explican por qué el tiempo importa.
- La validación temporal es una forma de estimar el error fuera de muestra bajo las condiciones reales de uso.

Slide 12

Leakage: cuando el futuro se filtra en el modelo

Un modelo solo debe usar información disponible en el momento de la predicción.
Si usamos datos del futuro, sobrestimamos su desempeño.

¿Qué es el leakage?

Ocurre cuando una variable o transformación contiene **información del futuro** respecto del momento de la predicción.

El modelo aprende patrones que no estarán disponibles en producción, por eso parece funcionar mejor de lo que realmente lo hará.

Ejemplos comunes

- Usar información del target o variables derivadas del futuro.
- Calcular estadísticas (media, tasa) con todo el dataset, incluyendo test.
- Usar variables que solo existen después del evento (ej.: resultados, aprobaciones, cierres).
- Transformaciones que miran hacia adelanto (ej.: rolling mal definido).

Ejemplo: predicción de churn

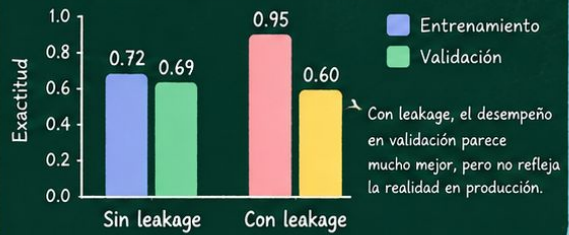
Problema:
Queremos predecir si un cliente cancelará el servicio al final del mes.

Dos conjuntos de variables:

Sin leakage (solo información pasada)		Con leakage (incluye información futura)	
Uso en los últimos 3 meses	✓	Pago del próximo mes	✗
Cantidad de reclamos	✓	Estado de cancelación	✗
Antigüedad del cliente	✓	Total gastado al final del año	✗
Plan contratado	✓	Resultado de una encuesta posterior	✗



Impacto en la validación



Con leakage, el desempeño en validación parece mucho mejor, pero no refleja la realidad en producción.

¿Cómo prevenirlo?

- ☑ Definir claramente el momento de la predicción.
- ☑ Usar solo información disponible hasta ese momento.
- ☑ Calcular transformaciones dentro de cada fold de validación.
- ☑ Hacer validación temporal cuando hay orden cronológico.
- ☑ Revisar que ninguna variable sea una consecuencia del target.

Puente P&E → ML

En estadística trabajamos con muestras **i.i.d. y sin sesgos.**

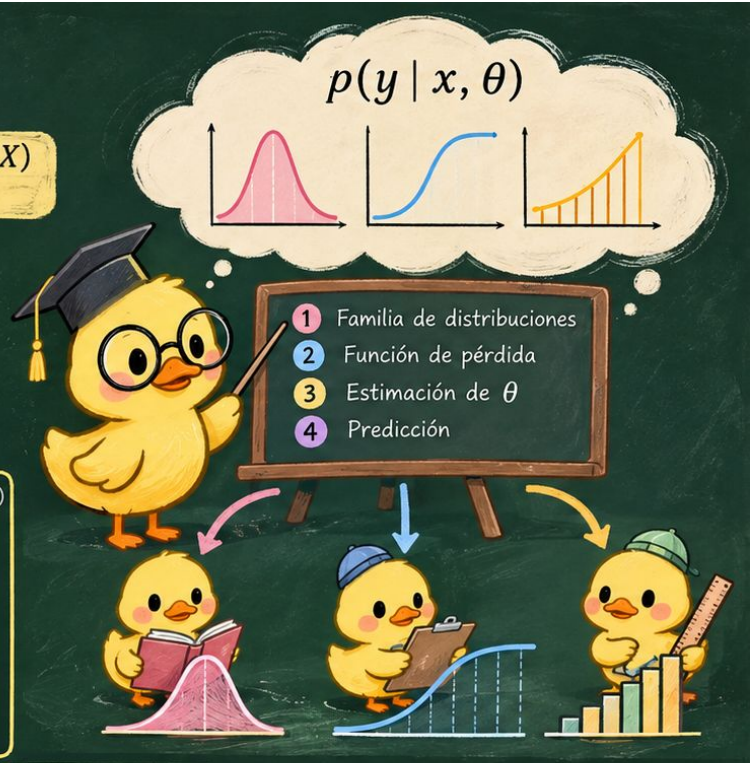
El leakage viola este supuesto, porque introduce información que no estaría **disponible para nuevas observaciones**, generando una estimación optimista del error.

13 Modelos probabilísticos; una misma idea, distintas tareas

Un modelo en ML define una distribución para Y (o para $Y \mid X$) con algunos parámetros θ, y los estima a partir de los datos.

Idea general

1. Elegimos una familia de distribuciones $p(y \mid x, \theta)$
2. Definimos una función de pérdida (derivada de la verosimilitud).
3. Estimamos θ con los datos (entrenamos).
4. Usamos el modelo para predecir o generar nuevas observaciones.



Tres ejemplos

Regresión (variable continua)

$Y \mid X \sim \mathcal{N}(\mu, \sigma^2)$

$\mu = \beta_0 + \beta_1 x$



Predice el valor esperado

$E[Y \mid X] = \mu$

Clasificación (variable discreta)

$Y \mid X \sim \text{Bernoulli}(p)$

$p = \sigma(\beta_0 + \beta_1 x)$



Predice una probabilidad

$P(Y = 1 \mid X) = p$

Conteo (eventos raros o frecuentes)

$Y \mid X \sim \text{Poisson}(\lambda)$

$\log(\lambda) = \beta_0 + \beta_1 x$



Predice la tasa esperada

$E[Y \mid X] = \lambda$

Interpretación

- Los parámetros θ controlan la forma de la distribución.
- Las predicciones incluyen **incertidumbre** (varianza).
- Un buen modelo tiene supuestos compatibles con los datos y con el objetivo del problema.

PUENTE P&E → ML

- Tal como en estadística elegimos una distribución para modelar un fenómeno y estimamos sus parámetros, en ML hacemos lo mismo para $Y \mid X$.
- La función de pérdida suele ser la **log-verosimilitud** (o una aproximación) y la estimación se realiza con MLE o variantes (MAP, regularización).

Verosimilitud Estadística

Mide qué tan compatibles son los datos observados con un modelo estadístico y sus parámetros.



1 Probabilidad vs. verosimilitud

Ambas parten de la misma expresión:

$$P(X = x \mid \theta)$$

Pero cambia su interpretación:

Concepto	Conocemos	Buscamos
Probabilidad (visión clásica)	Parámetros del modelo (θ)	Probabilidad de observar datos (x)
Verosimilitud (likelihood)	Datos observados (x)	Parámetros que mejor explican los datos (θ)

Probabilidad:

- Fijamos θ y variamos x.



Verosimilitud:

- Fijamos x y variamos θ.



$$L(\theta \mid x) = P(X = x \mid \theta)$$

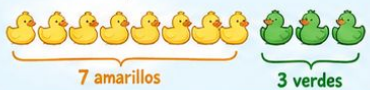
Para variables continuas se usa la función de densidad en lugar de una probabilidad puntual.

2 Ejemplo con patitos: distribución binomial

Una máquina produce patitos amarillos y verdes.
Desconocemos la proporción real de patitos amarillos, p.



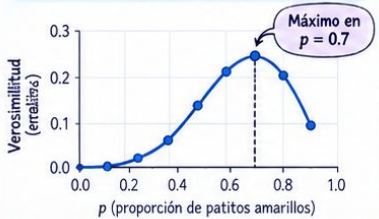
Observamos 10 patitos:



La verosimilitud binomial es:

$$L(p) = \binom{10}{7} p^7 (1-p)^3$$

Verosimilitud según la proporción de amarillos



La curva alcanza su máximo cuando:

$$\hat{p} = \frac{7}{10} = 0.7$$

El parámetro más verosímil es que el 70 % de los patitos sean amarillos.

Estimación por máxima verosimilitud (MLE):

$$\hat{\theta}_{MLE} = \arg\max_{\theta} L(\theta \mid x)$$

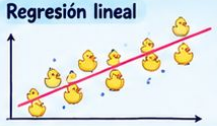
Importante: que $p = 0.7$ maximice la verosimilitud no significa que exista un 70 % de probabilidad de que ese parámetro sea el verdadero.



3 Relación con Machine Learning

Muchos modelos de ML se entrenan buscando los parámetros que maximizan la verosimilitud de los datos (o minimizando su negativo).

Regresión lineal



- Suponemos errores gaussianos independientes.
- Minimizar MSE equivale a **maximizar la verosimilitud.**

$$y = \beta_0 + \beta_1 x + \varepsilon \qquad \varepsilon \sim N(0, \sigma^2)$$

Regresión logística



- Modelo Bernoulli.
- Minimizar binary cross-entropy equivale a maximizar la log-verosimilitud.

$$P(y = 1 \mid x) = \sigma(\beta_0 + \beta_1 x)$$

Naive Bayes



- Utiliza verosimilitudes de las características condicionadas a cada clase.
- Combinadas con probabilidades a priori mediante Bayes.

4 Log-verosimilitud y función de pérdida

Log-verosimilitud:

$$\ell(\theta) = \log L(\theta \mid x)$$

- Es más estable numéricamente.
- Convertir productos en sumas.

Función de pérdida en ML:

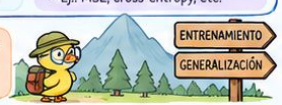
$$Loss = -\log L(\theta \mid x)$$

- Muchos algoritmos minimizan el negativo de la log-verosimilitud.
- Ej.: MSE, cross-entropy, etc.

El puente entre Estadística y ML

- En estadística buscamos parámetros que **expliquen** los datos observados.
- En ML usamos esa misma idea para ajustar modelos mediante **funciones de pérdida.**

! Maximizar la verosimilitud sobre los datos de entrenamiento no garantiza que el modelo generalice bien a datos nuevos. Necesitamos validación (e.g., datos de test, cross-validation).



14 Entrenar = máxima verosimilitud

Aprender un modelo es estimar los parámetros que hacen más probables los datos observados.

1 Planteamos un modelo probabilístico

- Asignamos una distribución a $Y \mid X$ con parámetros θ.
- Ejemplo (regresión lineal):

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2)$$

2 Escribimos la verosimilitud

- La probabilidad de observar todos los datos $\{(x_i, y_i)\}_{i=1}^n$ como función de θ.
- Para el ejemplo:

$$L(\beta_0, \beta_1, \sigma^2) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \beta_0 - \beta_1 x_i)^2}{2\sigma^2}\right)$$

3 Optimizamos (equivalente a minimizar una pérdida)

- Maximizamos L o, más fácil, su logaritmo.
- En este caso es equivalente a minimizar el Error Cuadrático Medio (MSE):

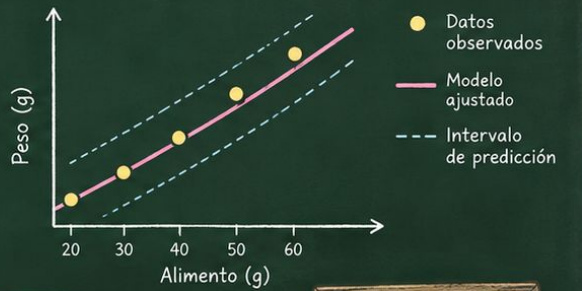
$$\hat{\beta}_0, \hat{\beta}_1 = \arg\min_{\beta_0, \beta_1} \frac{1}{n} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2$$

4 Obtenemos predicciones e incertidumbre

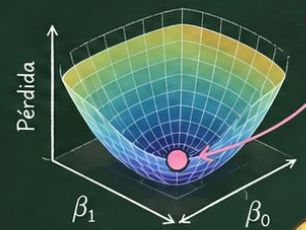
- Predicción puntual: $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$.
- Distribución de la predicción: $\hat{Y} \mid X = x \sim \mathcal{N}(\hat{\beta}_0 + \hat{\beta}_1 x, \sigma^2)$.
- Podemos construir intervalos de predicción: $\hat{y} \pm 1{,}96 \cdot \sigma$ (aprox. 95%)

Ejemplo con datos de patitos

Alimento (g)	Peso (g)
20	420
30	455
40	475
50	520
60	535



Paisaje de la función de pérdida





Puente P&E → ML



- Estimar parámetros de una distribución (P&E) ↔ entrenar un modelo (ML).
- La máxima verosimilitud conecta ambos: elegir los parámetros que hacen más probables los datos.
- La función de pérdida es (en muchos modelos) la log-verosimilitud con otro nombre.
- Las predicciones van acompañadas de incertidumbre, igual que en inferencia estadística.

15 Modelo lineal: ¿qué supone y qué estima?

La idea

El modelo lineal relaciona X con Y mediante una combinación lineal de variables:

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \varepsilon$$

donde ε es un error con media 0 y varianza σ^2.

Supuestos principales

1. Relación lineal en los parámetros.
2. Errores con media 0 y varianza constante (homoscedasticidad).
3. Errores independientes.
4. Sin multicolinealidad perfecta.
5. (Para inferencia) Errores aproximadamente normales.

¿Qué estima?

Encuentra los coeficientes $\hat{\beta}$ que minimizan la suma de errores cuadráticos (MSE):

$$\hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

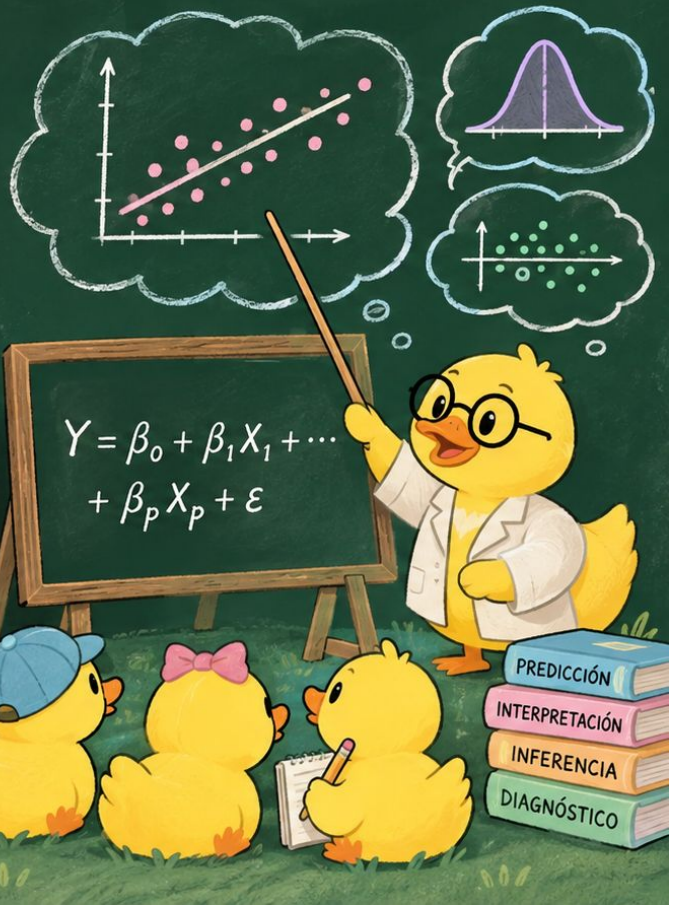
Equivale a máxima verosimilitud si $\varepsilon \sim \mathcal{N}(0, \sigma^2)$.

¿Qué nos da?

- Predicciones: $\hat{y} = X\hat{\beta}$.
- Interpretación: cada β_j es el cambio promedio en Y por una unidad de X_j, manteniendo las demás constantes.
- Medidas de ajuste: MSE, R^2.
- Incertidumbre: errores estándar e intervalos para los coeficientes.

Puente P&E → ML

Estimación de parámetros (OLS) → - entrenar un modelo.
Supuestos del modelo → - hipótesis sobre la distribución de los errores.
Inferencia de β → - interpretar y cuantificar incertidumbre.
Diagnóstico de residuos → validar si los supuestos son razonables.



Slide 16

Modelos de conjunto: muchos modelos, una mejor predicción

Idea central

- Combinan muchos modelos simples para reducir error y mejorar la generalización.
- Cada modelo aprende un patrón distinto de los datos.
- Al combinarlos, se reduce la **varianza**, el **sesgo** o ambos.

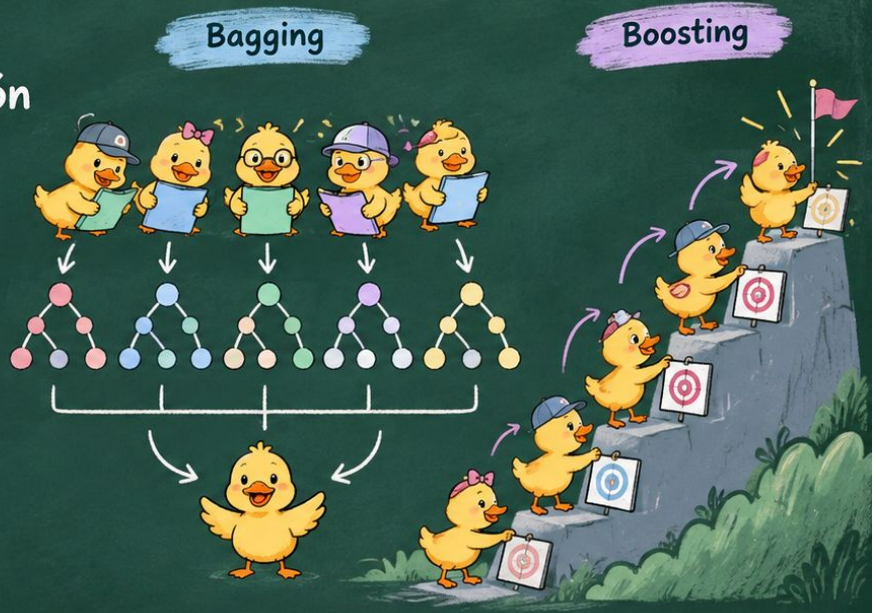
Principales enfoques

- **Bagging** (ej. Random Forest): entrena muchos modelos en muestras distintas y promedia sus predicciones.
- **Boosting** (ej. Gradient Boosting): entrena secuencialmente, cada modelo corrige los errores del anterior.
- **Resultado:** menor error de generalización que un modelo individual (típicamente baja la varianza y puede reducir el sesgo).

Ejemplo intuitivo

Predecimos si un patito tendrá sobrepeso (sí/no) usando 5 árboles.
Cada árbol mira una muestra distinta de los datos y votan.
La predicción final es la clase con más votos.

- **Predicciones de los árboles:** Sí, No, Sí, Sí, No → **Predicción final:** Sí



Puente P&E → ML

- **Combinación de estimadores:** el promedio de varios modelos reduce la varianza (para modelos poco correlacionados).
- Bagging ≈ bootstrap + agregación de estimadores.
- Boosting ≈ construcción secuencial que reduce el sesgo.
- Se apoya en los conceptos de **sesgo**, **varianza** y error de generalización.

17 Validación = inferencia sobre el error

Evaluamos si el modelo generaliza y si la mejora es real.

1 Estimamos el error de generalización

- Dividimos los datos: train / valid (o cross-validation).
- La métrica en valid es un estimador del error en datos nuevos.

2 Cuantificamos la incertidumbre

- Usamos el TCL (o bootstrap) para construir intervalos de confianza de la métrica.
- Ejemplo: exactitud = 0.82 con IC 95% [0.78, 0.86].

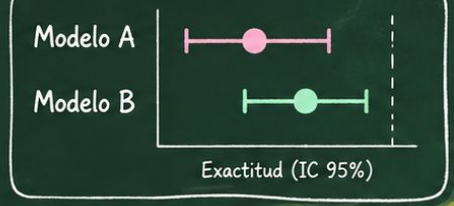
3 Comparamos modelos como un test de hipótesis

- H_0: no hay diferencia real entre modelos.
- Si los intervalos se superponen mucho, la mejora puede ser solo ruido.

4 Errores Tipo I y Tipo II en ML

- Tipo I: declarar "mejor" un modelo que no lo es.
- Tipo II: no detectar una mejora real.
- Elegimos el riesgo aceptable según el problema.

Ejemplo: comparar dos modelos

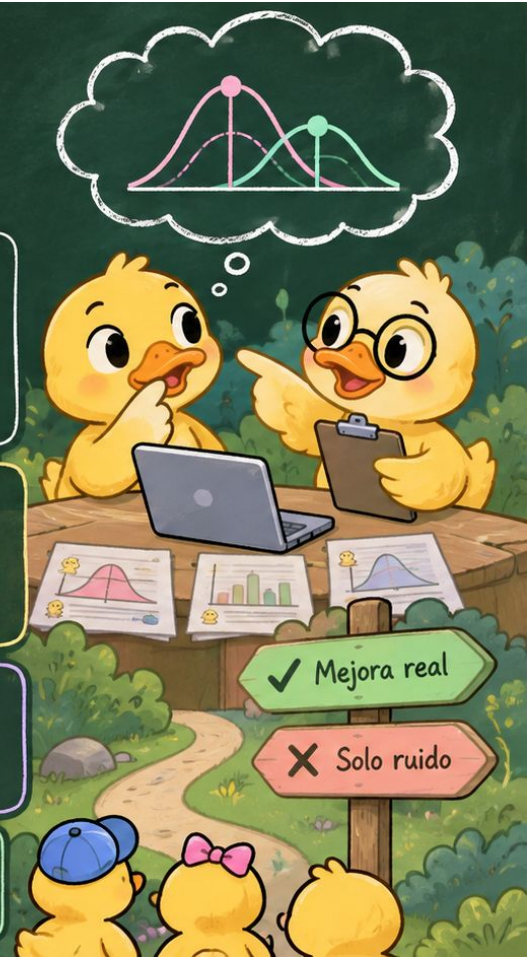


Interpretación

- Si los IC se solapan: no hay evidencia de que uno sea mejor.
- Si no se solapan: la diferencia es estadísticamente significativa.

Otras métricas

- Clasificación: exactitud, F1, AUC.
- Regresión: MAE, RMSE.
- Reportar siempre la incertidumbre (IC o bootstrap).



Puente P&E → ML

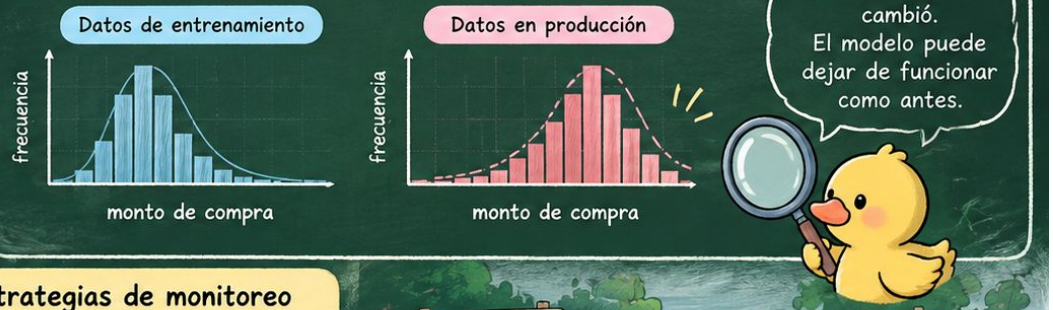
En estadística, usamos test de hipótesis e intervalos para decidir si un efecto es real. → En ML, usamos las mismas ideas para decidir si un modelo generaliza mejor y por cuánto.

18. De la validación a la producción: drift, monitoreo y decisiones

¿Qué puede cambiar?

- **P(X):** las características de los datos.
  Ej.: cambia la distribución de edades.
- **P(Y | X):** la relación entre **X e Y.**
  Ej.: cambia el comportamiento de los clientes.
- **Proceso de predicción:** diferencias entre entrenamiento y producción.
  Ej.: otras reglas de negocio o datos incompletos.

Ejemplo: drift en la distribución de una variable

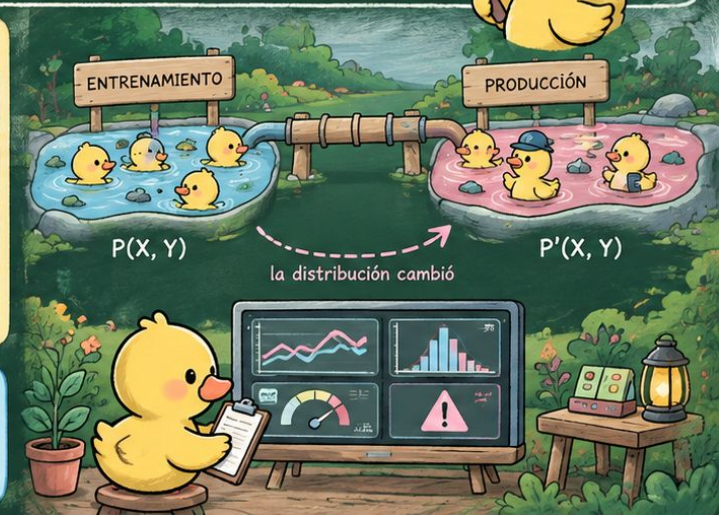


Tipos de drift

Covariate shift	Cambia P(X), pero P(Y | X) se mantiene.
Concept shift	Cambia P(Y | X). La relación entre variables ya no es la misma.
Training-serving skew	Los datos en producción difieren de los usados en entrenamiento (por procesos, reglas o latency).

Estrategias de monitoreo

- Monitorear la distribución de variables clave.
- Seguir métricas de performance en el tiempo.
- Definir umbrales y alertas.
- Reentrenar o ajustar cuando se detecta drift.



Puente P&E → ML

En estadística vimos que una estimación vale bajo ciertos supuestos sobre la población. En producción, esos supuestos pueden dejar de cumplirse.
Drift = cambio en la distribución = violación de supuestos.
Monitorear es verificar que el modelo siga siendo válido para la nueva distribución.

19 Tipos de datos: cómo define qué variable aleatoria tenés

1. Estructurado

Filas y columnas con tipos claros.

id	edad	ingresos	compró
1	28	1200	1
2	35	950	0
3	42	2000	1
...	...	...	...

- Variables numéricas, categóricas, binarias, fechas.
- Representa variables aleatorias de soporte finito o continuo.

2. Semi-estructurado

Tiene una estructura flexible.

```
{
   "usuario": 123,
   "dispositivo": "móvil",
   "ubicación": {
      "lat": -34.6,
   }   "lon": -58.4
}
```

- Ejemplos: JSON, XML, logs, datos de sensores.
- Contiene variables anidadas, listas o de longitud variable.
- Se modela como variables aleatorias estructuradas.

3. No estructurado

Sin formato fijo, alta complejidad.



- Ejemplos: texto libre, imágenes, audio, video.
- Representa variables aleatorias de alta dimensión (píxeles, palabras, frecuencias).
- Requiere representaciones especiales (embeddings, features aprendidas).

¿Qué tareas y modelos son típicos?

- **Clasificación / regresión** (tabular clásica)
- **Modelos:** regresión lineal, logística, árboles, ensembles.
- **Ejemplos:** scoring, churn, demanda.

- **Clasificación / regresión**
- **Detección de patrones**
- **Modelos:** árboles, gradient boosting, modelos secuenciales.
- **Ejemplos:** análisis de logs, IoT, clickstream.

- NLP (texto)
- Visión por computadora (imágenes)
- **Audio** (voz, sonidos)
- **Modelos:** redes neuronales, transformers, modelos generativos.
- **Ejemplos:** análisis de sentimiento, detección de objetos, transcripción.





Puente P&E → ML

- El tipo de dato define qué espacio muestral tiene la variable aleatoria X.
- La tarea define qué distribución o función estimamos (P(Y | X), E[Y | X], densidad, etc.).
- El área de ML determina qué supuestos y representaciones son razonables.

20 Detección de anomalías
valores atípicos y baja densidad



1. ¿Qué es?

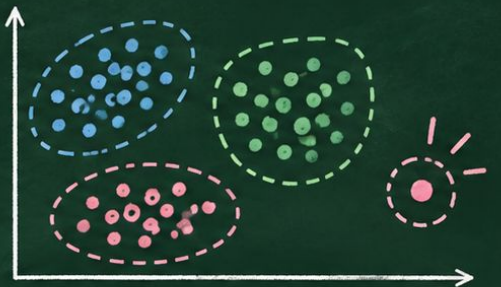
Una anomalía es una observación poco probable bajo el comportamiento normal de los datos.
Puede ser un valor atípico en una variable o una combinación rara de varias variables.

2. ¿Por qué importa?

Permite detectar fraudes, fallas, eventos inusuales o comportamientos nuevos que requieren atención.

3. Enfoques principales

- Univariado: valores atípicos por IQR o z-score.
- Multivariado: baja densidad en el espacio de variables (p. ej. distancia, clustering, Isolation Forest).
- Por aprendizaje: autoencoders, One-Class SVM.



4. Ejemplo simple

Peso de los patitos (g).
Media = 500, desvío = 50.
Un patito de 700 g tiene $z = 4 \rightarrow$ valor atípico.

5. Consideraciones

- No toda anomalía es un error (puede ser un caso real).
- Validar con conocimiento del dominio.



PUENTE P&E → ML

Un valor atípico es una observación en la cola de la distribución. En ML, la anomalía suele definirse como baja densidad o gran distancia a los datos normales.

21. NLP: del texto al significado

Los textos también son datos: cómo representarlos y modelarlos.

Problema

Clasificar el sentimiento de opiniones de patitos sobre un producto: positivo, neutro o negativo.

Datos

- Textos cortos (reseñas).
- Etiqueta de sentimiento (positivo, neutro, negativo).
- Ejemplo: "Me encantó el alimento, mis patitos están felices" → positivo.

Análisis estadistico

- Limpieza: minúsculas, sin signos, sin palabras vacías.
- Exploración: frecuencia de palabras, nubes, n-gramas.
- Representación: Bag of Words, TF-IDF o embeddings.
- Revisión de balance de clases y longitud de textos.

Decisión de ML

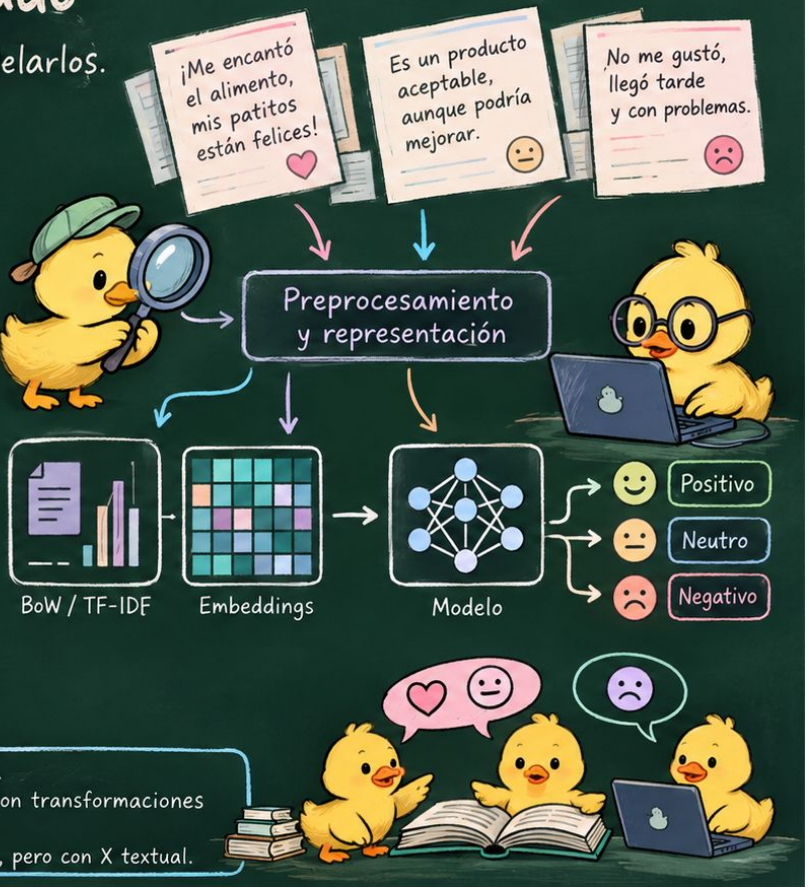
- Modelos: regresión logística, SVM, redes neuronales (embeddings + fine-tuning).
- La entrada es una representación numérica del texto: vector de frecuencias o embedding denso.
- El modelo estima P(Y = sentimiento | texto).

Validación

- Métricas: exactitud, precisión, recall, F1 (por clase).
- Validación cruzada.
- Analizar errores: sarcasmo, negaciones, ambigüedad.

Puente P&E → ML

- El texto es una variable aleatoria X (secuencia de palabras).
- Las representaciones (frecuencias, TF-IDF, embeddings) son transformaciones de X que conservan información relevante.
- El modelo estima P(Y | X) igual que en clasificación clásica, pero con X textual.



22 LLM: lenguaje, probabilidad y generación de texto

¿Cómo se predice la siguiente palabra?

1 Idea central

- Un LLM modela la probabilidad de la siguiente palabra dada una secuencia: $P(w_t \mid w_1, \ldots, w_{t-1})$.
- Se entrena para maximizar la verosimilitud de los textos.
- Luego genera texto eligiendo palabras según esa distribución.

2 Datos y representación

- Texto → secuencia de tokens (palabras o subpalabras).
- Cada token se convierte en un vector (embedding).
- La secuencia se procesa con una arquitectura Transformer que captura relaciones de largo alcance.

3 Objetivo de entrenamiento

Maximizar la verosimilitud (o minimizar la pérdida):

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(w_t \mid w_1, \ldots, w_{t-1})$$

- Equivale a minimizar la entropía cruzada entre el texto real y el modelo.
- Se aprende con técnicas de optimización (gradiente descendente).

4 Generación de texto

- Dado un contexto, se calcula $P(w_t \mid \text{contexto})$.
- Se elige la siguiente palabra (por ejemplo, con muestreo, top-k o temperatura).
- Se repite el proceso para completar la secuencia.

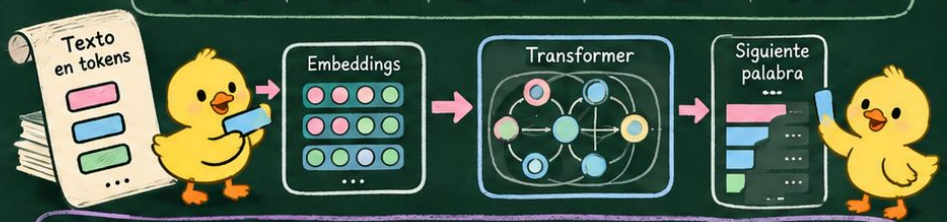
Ejemplo: predicción de la siguiente palabra

Contexto	Distribución $P(w_t \mid \text{contexto})$	
Los patitos	nadan	0,55
	comen	0,25
	duermen	0,10
	...	
Los patitos nadan	en	0,60
	rápido	0,20
	juntos	0,10
	...	



Aplicaciones

- Asistentes virtuales
- Resumen de textos
- Traducción automática
- Generación de contenido
- Búsqueda semántica



Puente P&E → ML

- Probabilidad condicional → modelo de lenguaje $P(w_t \mid \text{contexto})$.
- Estimación de parámetros → maximización de verosimilitud en grandes textos.
- Inferencia → generación y evaluación de texto en tareas reales.
- Supuestos → dependencia entre tokens (no i.i.d.), modelada con Transformers.

23. CASO DE USO I

Fraude en transacciones

Cómo pasamos de datos crudos a un modelo que detecta operaciones sospechosas.

1. Problema y objetivo

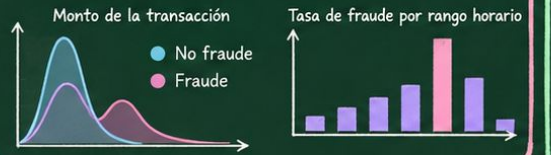
- Identificar transacciones fraudulentas en tiempo real.
- **Objetivo:** estimar P(Y = fraude | X) para bloquear, revisar o aprobar.

2. Datos y contexto

- Datos: transacciones con monto, comercio, ubicación, dispositivo, hora, historial del usuario.
- Variable objetivo: fraude (sí/no).
- Particularidades:
  - clases muy desbalanceadas (fraude < 1%),
  - variables categóricas de alta cardinalidad,
  - patrones que cambian en el tiempo.

3. Análisis exploratorio (P&E)

- Tasa de fraude según variables clave.
- Distribuciones de monto y hora.
- Relaciones entre variables (correlación, mutua información).

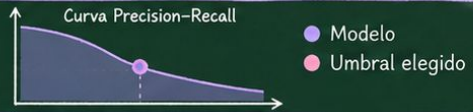


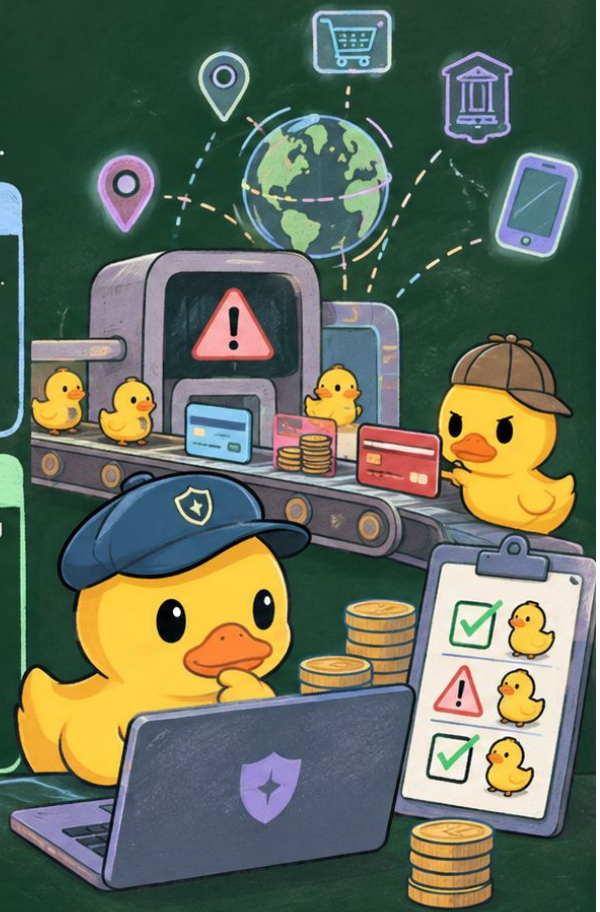
4. Decisión de Machine Learning

- Modelo: regresión logística, árboles con boosting o modelos de ensamblado.
- Features clave:
  - transformaciones de monto (log),
  - frecuencias por comercio (con suavizado),
  - variables de comportamiento reciente.
- Manejamos el desbalanceo con:
  - ponderación de clases,
  - submuestreo / sobremuestreo,
  - o umbrales ajustados al costo de errores.

5. Validación e interpretación

- Métricas adecuadas para clases desbalanceadas:
  - curva Precision–Recall y su área,
  - matriz de confusión a un umbral elegido,
  - análisis de costo: ¿cuánto cuesta un falso negativo vs. un falso positivo?





CASO 2

Scoring de crédito: de los datos al modelo

Predecir la probabilidad de que un cliente no pague su préstamo, combinando análisis estadístico, ingeniería de features y un modelo probabilístico.

Objetivo didáctico

- Aplicar todo el puente P&E → ML en un caso real.
- Ver cómo cada decisión se justifica con evidencia de los datos.
- Interpretar resultados y validar el modelo.

1. Problema y datos

¿Qué queremos predecir?

La probabilidad de que un cliente no pague en los próximos 12 meses.

Datos disponibles

- Información del cliente (edad, ingresos, historial).
- Variables del préstamo (monto, plazo, tasa).
- Comportamiento de pago (en los últimos meses).



2. Análisis estadístico (EDA)

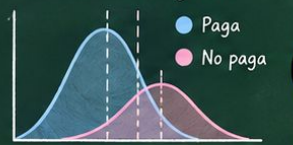
Variable objetivo

Tasa de no pago: 8%

Variables clave

- **Ingresos:** menor en clientes que no pagan.
- **Historial de pago:** más atrasos en clientes que no pagan.
- **Monto del préstamo:** más alto en clientes que no pagan.

Distribución de ingresos



3. Ingeniería de features

Variables transformadas:

- Ratio deuda/ingreso.
- Cantidad de atrasos (últimos 12m).
- Antigüedad del cliente.
- Tipo de producto (one-hot).
- Interacciones relevantes (p. ej. monto x plazo).



4. Modelo: regresión logística

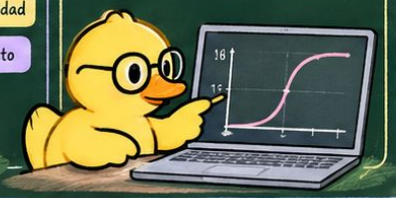
Estimamos la probabilidad de no pago como:

$$P(Y=1 \mid X) = \sigma(\beta_0 + \beta^T X)$$

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Interpretación:

- Cada coeficiente indica cómo una variable afecta la probabilidad de no pago, manteniendo las demás constantes.
- Se entrena por máxima verosimilitud (equivale a minimizar la log-loss).

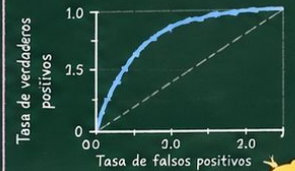


5. Validación y resultados

Desempeño en test

AUC:	0,82
Precisión:	0,76
Recall:	0,68
F1:	0,72

Curva ROC



Conclusiones

- El modelo discrimina bien (AUC 0,82).
- Las variables más importantes coinciden con la intuición del negocio.
- Listo para producción con monitoreo de drift y performance.



Puente P&E → ML

EDA = estadística descriptiva para entender la distribución de X e Y → Feature engineering = transformaciones de variables aleatorias → Entrenar = estimar parámetros por máxima verosimilitud → Validar = inferencia sobre el error de generalización

25 Caso 3 – Fraude en transacciones

¿Cómo detectamos operaciones sospechosas sin generar muchas falsas alarmas?

1 Problema y objetivo

Detectar transacciones fraudulentas en tiempo real.
El fraude es poco frecuente y costoso: si no lo detectamos, la pérdida es alta; si bloqueamos de más, afectamos a clientes reales.

2 Datos

- Cada fila: una transacción con monto, comercio, ubicación, dispositivo, hora y características del cliente.
- Variable objetivo: fraude (sí / no).
- Desbalance: por ejemplo, 0,5% de transacciones son fraude.

Ejemplo simplificado

Monto	Hora	País	Fraude
120	14:32	AR	0
980	02:11	BR	1
45	16:20	AR	0
300	03:47	AR	1

3 Análisis estadístico (EDA)

- El fraude es raro: la tasa global es 0,5%.
- Los fraudes tienen, en promedio, montos más altos y ocurren más de noche.
- Algunas categorías (comercio, país) muestran tasas de fraude distintas.
- Hay correlación temporal: varios intentos en poco tiempo.



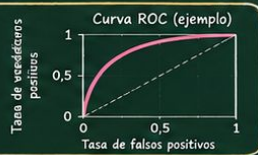


4 Decisión de Machine Learning

- Tarea: clasificación binaria con desbalance.
- Modelo: regresión logística, árboles (Gradient Boosting) o modelos de ensamble.
- Features: transformaciones del monto (log), variables temporales (hora, día), conteo de transacciones recientes, target encoding con suavizado.
- Estrategia: ponderar clases o re-muestrear para manejar el desbalance.

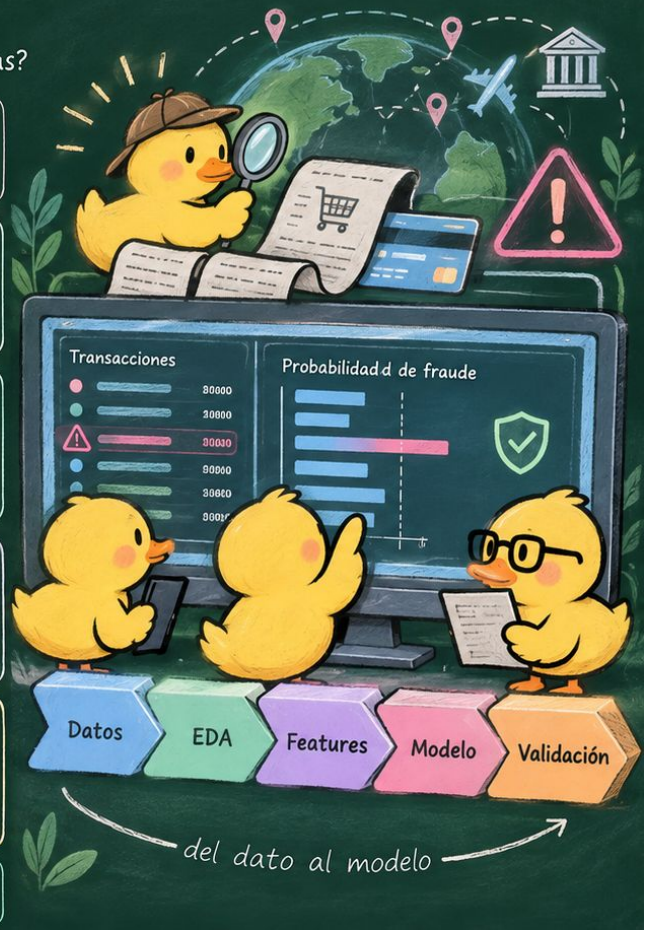
5 Validación y resultados

- Métrica principal: curva ROC y AUC. También precision-recall por el desbalance.
- Elegir un umbral según el costo de error (pérdida por fraude vs. cliente molesto).
- Validación temporal: entrenar con datos de meses anteriores y probar con meses futuros.
- Ejemplo de resultado: AUC = 0,95. Con un umbral que da recall = 0,80, precision = 0,60 (se detecta 80% de fraudes, con 40% de falsas alarmas).



Puente P&E → ML

- Distribución y desbalance → estimación de probabilidades $P(Y = 1 \mid X)$.
- Test de hipótesis → elección de umbral y control de errores (Tipo I: falsa alarma, Tipo II: fraude no detectado).
- Series temporales → validación temporal para evitar leakage.
- Inferencia → intervalos de confianza para métricas y monitoreo de drift en producción.



26 Ciberseguridad: convertir logs en señales de anomalía

1 Problema

Detectar comportamientos inusuales en la red para identificar posibles ataques y reducir falsos positivos.

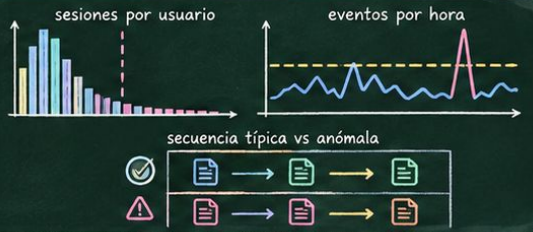
2 Datos

- Logs de eventos de red por usuario o dispositivo.
- Variables: hora, IP origen, tipo de evento, bytes, sesión, geo, etc.
- Datos desbalanceados: muy pocas anomalías.

tiempo	usuario	evento	bytes
...	...	...	...
...	...	...	...
...	...	...	...

3 Análisis estadístico (EDA)

- Distribuciones: sesiones por usuario, bytes por hora, duración.
- Patrones temporales: picos inusuales.
- Relaciones: ciertos eventos ocurren en secuencia.
- Variables categóricas de alta cardinalidad: IP, país, tipo de dispositivo.



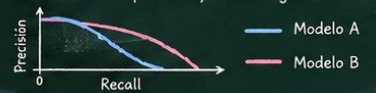
4 Decisión de ML

- Construir features que capturen comportamiento:
  - frecuencia de eventos en ventanas de tiempo,
  - estadísticos por usuario o IP (media, varianza),
  - ratios (bytes por sesión, conexiones fallidas / totales),
  - embeddings de secuencias (opcional).
- Modelos según objetivo:
  - clasificación (logs etiquetados) → Regresión logística, árboles, gradient boosting.
  - detección de anomalías (pocos o sin etiquetados) → Isolation Forest, One-Class SVM, autoencoders.



5 Validación

- Separación temporal: entrenar con logs pasados, validar con período posterior.
- Evaluar con métricas robustas al desbalance:
  - Precision, Recall, F1
  - curva Precision-Recall
- Analizar falsos positivos y falsos negativos.



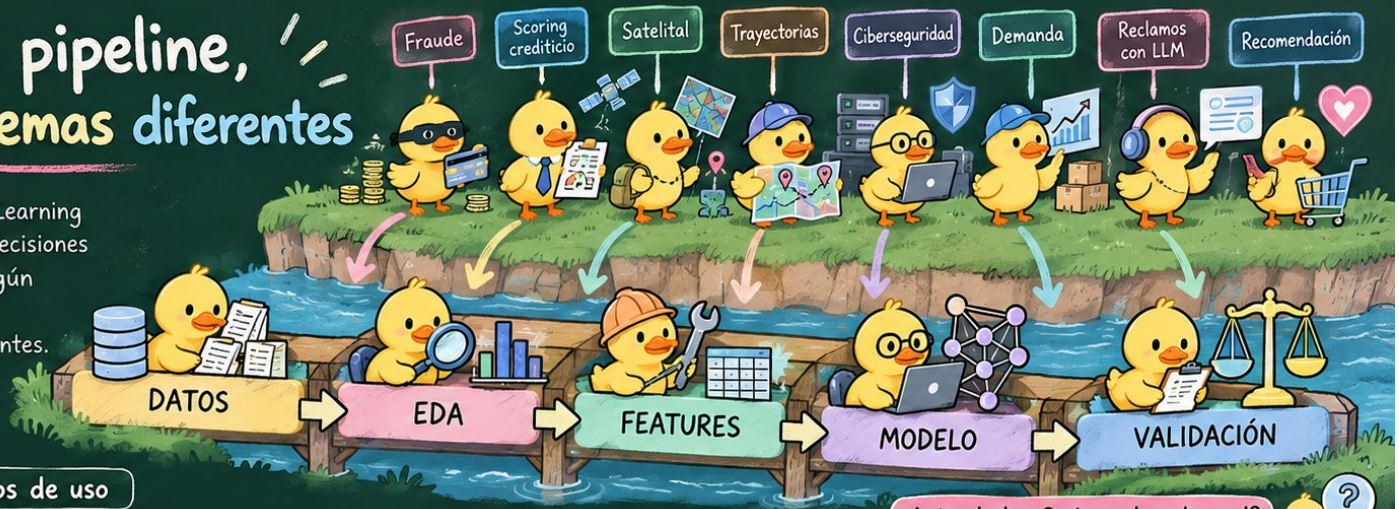


Puente P&E → ML

- EDA = estadística descriptiva de procesos de eventos.
- Features = resumen de una distribución o dependencia temporal.
- Modelos = estiman P(anomalía | X) o detectan baja densidad.
- Validación = inferencia sobre la capacidad de generalizar a eventos futuros.

Un mismo pipeline, ocho problemas diferentes

El proceso de Machine Learning es el mismo, pero las decisiones estadísticas cambian según el tipo de dato, la tarea y las dependencias presentes.



Comparación de casos de uso

Caso	Tipo de dato	Tarea	P&E clave	Features posibles	Validación
Fraude	Transacciones	Clasificación	Poisson, colas	Frecuencia, monto atípico	Temporal
Scoring crediticio	Tabular	Clasificación	Bayes, log-odds	Deuda/ingreso, WoE	Temporal y por cohorte
Satelital	Imágenes	Clasificación	Autocorrelación espacial	NDVI, bandas	Espacial
Trayectorias	Secuencias	Predicción	Procesos temporales	Velocidad, aceleración	Por trayectoria
Ciberseguridad	Logs	Detección de anomalías	Entropía, Poisson	Conexiones/min, puertos	Temporal y por host
Demanda	Series de tiempo	Predicción	ACF, estacionalidad	Lags, rolling mean	Temporal
Reclamos con LLM	Texto	Clasificación	Probabilidad condicional	TF-IDF, embeddings	Por documento/grupo
Recomendación	Interacciones	Rankeo	P(B|A), sesgo	Historial, recencia	Temporal y por usuario

Actividad: ¿Qué puede salir mal?

1. **Dependencia**
   ¿Qué observaciones no son independientes entre sí?
2. **Feature Engineering**
   ¿Qué estadístico transformarías en una variable predictora?
3. **Validación**
   ¿Qué sucedería si separaras train/test de forma aleatoria?

Puente P&E → ML

- La estadística determina cómo interpretar los datos, construir variables y estimar la incertidumbre.
- Generalizar no significa únicamente obtener una buena métrica: significa que esa métrica sea representativa del escenario donde se utilizará el modelo.

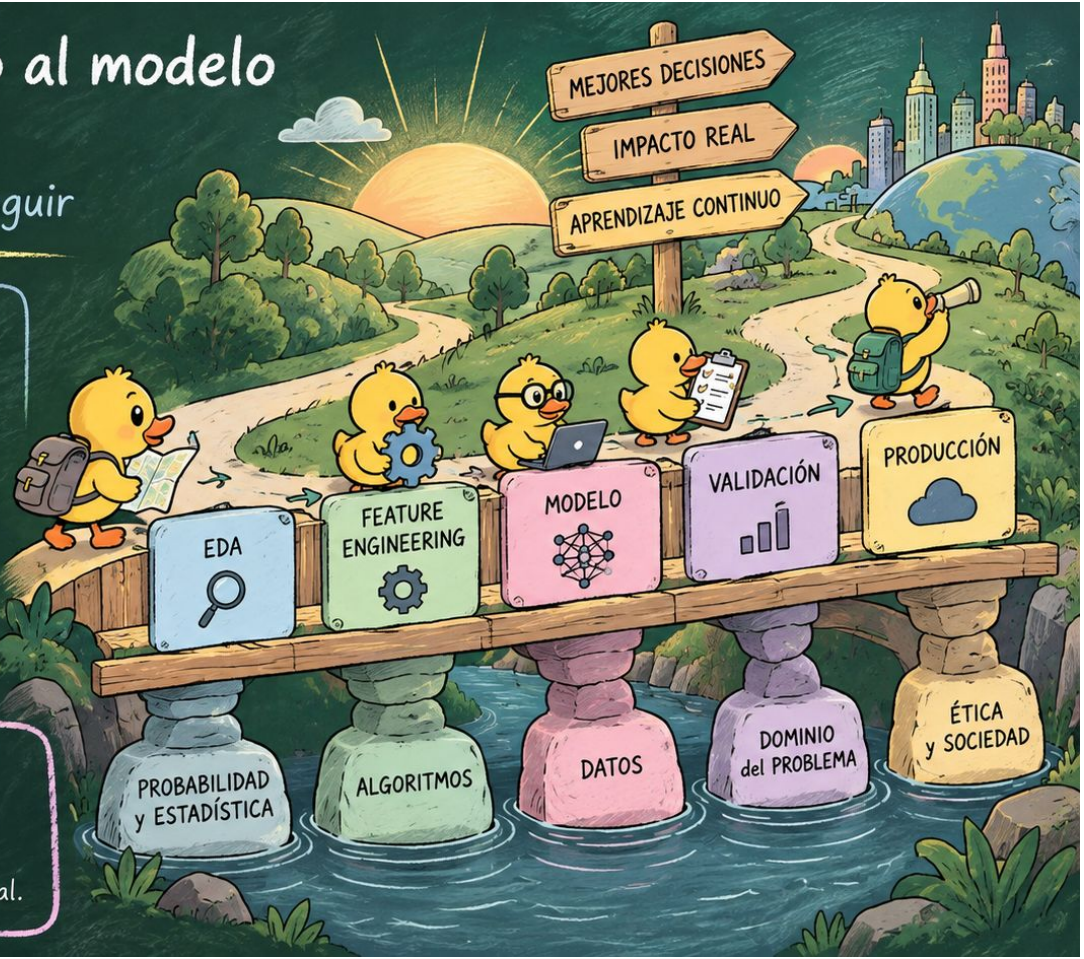
28 Cierre: del dato al modelo y más allá

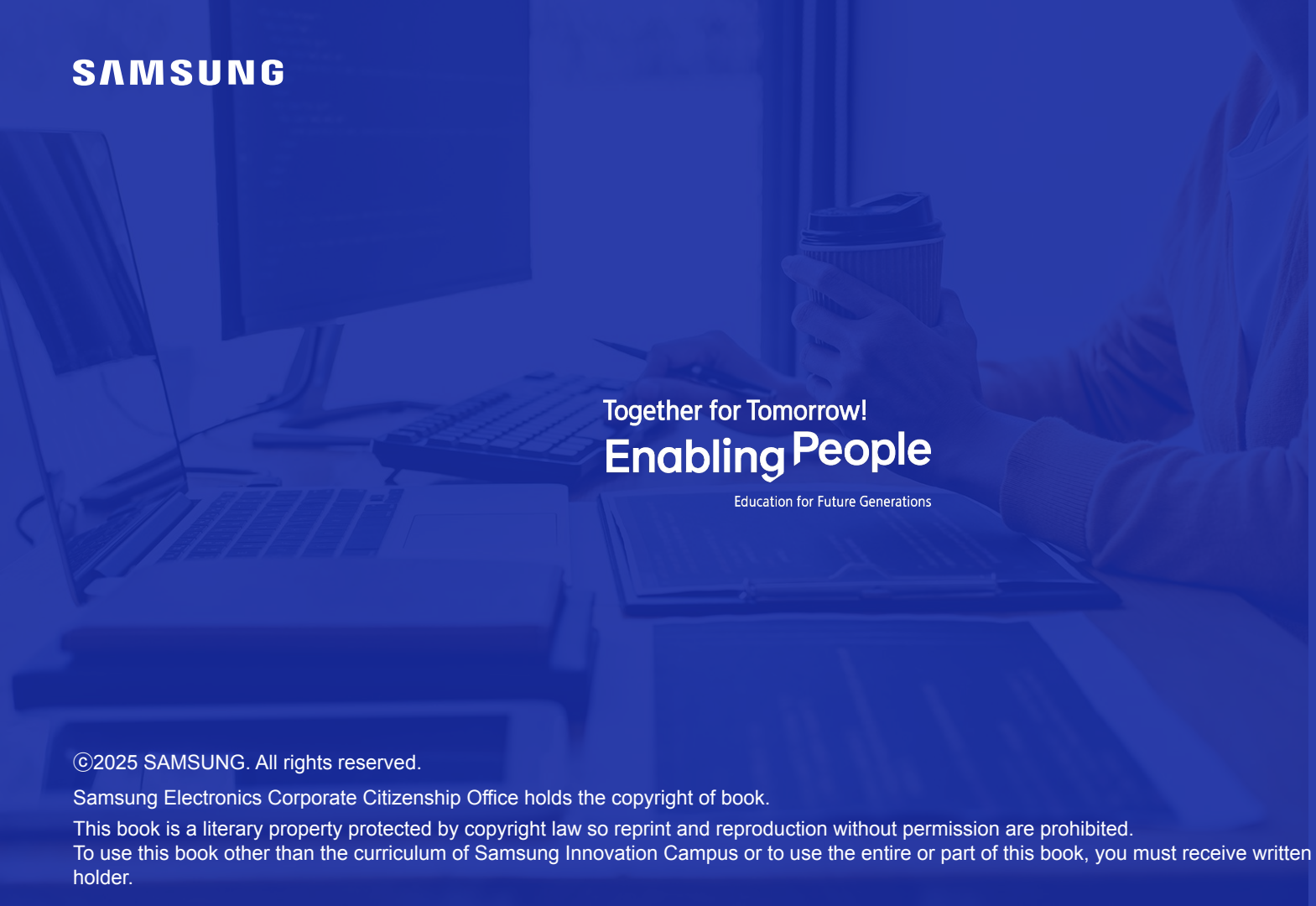
Lo que integramos y cómo seguir

- ☑ Leer un dataset como una muestra.
- ☑ Justificar cada feature con EDA.
- ☑ Explicar los supuestos probabilísticos.
- ☑ Validar como un problema de inferencia.
- ☑ Detectar leakage y drift.
- ☑ Ubicar cualquier caso según dato, tarea y área.

Idea clave

Machine Learning no es una caja negra: es estadística aplicada con propósito, desde los datos hasta decisiones en el mundo real.





ⓒ2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of book.

This book is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this book other than the curriculum of Samsung Innovation Campus or to use the entire or part of this book, you must receive written consent from copyright holder.